

Legal, Ethical, Professional, and Social Issues in Software Engineering

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

08 May 2026

Today's Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections
- 6 Summary & Conclusion

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections
- 6 Summary & Conclusion

Revisiting LEPSI

In Week 3, we introduced **LEPSI** (Legal, Ethical, Professional, and Social Issues) as strict Non-Functional Requirements (NFRs).

You learned that things like GDPR, CCPA, and HIPAA dictate how you design your databases. You also learned about WCAG accessibility standards.

Today, we move beyond basic compliance. We will explore the complex ethical realities of modern software engineering, intellectual property, and provide a structured framework for writing academic reflections on your engineering decisions.

The Professional Code of Conduct

Software Engineering is a recognized profession. Like doctors or civil engineers, we are bound by Codes of Conduct, such as the ACM/IEEE-CS Code of Ethics.

ACM Code of Ethics & Professional Practice

Key principles include:

- **Public:** Software engineers shall act consistently with the public interest.
- **Product:** Ensure that products meet the highest professional standards possible.
- **Judgment:** Maintain integrity and independence in professional judgment.
- **Colleagues:** Be fair to and supportive of colleagues.

ACM Code of Ethics and Professional Conduct

[Home](#) > [Code Of Ethics](#) > [The Software Engineering Code Of Ethics And Professional Practice](#)

The Software Engineering Code of Ethics and Professional Practice

Software Engineering Code of Ethics and Professional Practice (Version 5.2) as recommended by the ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices and jointly approved by the ACM and the IEEE-CS as the standard for teaching and practicing software engineering.

The joint ACM/IEEE-CS Software Engineering Code was published as: Don Gotterbarn, Keith Miller, and Simon Rogerson. 1997. Software engineering code of ethics. *Commun. ACM* 40, 11 (November 1997), 110-118. DOI: [10.1145/265684.265699](https://doi.org/10.1145/265684.265699)

Note that this code is for anyone that is a member of the software engineering profession, regardless of ACM membership status. You may also wish to consult [The Code for all ACM members](#) (regardless of profession).

- [ACM Code of Ethics](#)
- [Report a Potential Code Violation](#)
- [Using the Code](#)
- [Case Studies](#)
- [Previous Versions of the ACM Code](#)
- [Other Codes](#)
- [The Code 2018 Update Project](#)
- [About ACM's Committee on Professional](#)

`https://www.acm.org/code-of-ethics/
software-engineering-code`

When Professionalism Fails: Volkswagen

What happens when technical constraints override ethical judgment?

The Volkswagen Emissions Scandal (2015)

- Volkswagen intentionally programmed diesel engine vehicles to activate emission controls *only* during laboratory testing.
- On the road, vehicles output more than 40 times the allowed NOx limits.
- **Why?** Engineers were unable to meet EPA requirements in the allotted time and budget. Faced with a toxic work culture, they opted to rig the software test rather than fix the engineering problem.
- **Result:** It cost the company over 31 billion euros and led to arrests of top executives.

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing**
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections
- 6 Summary & Conclusion

Intellectual Property (IP) in Software

Intellectual property refers to creations of the mind. IP rights are crucial to protecting software and code.

- **Copyrights:** Protect original works of authorship, such as artwork, software code and documentation.
- **Patents:** Protect inventions, such as new software algorithms or processes.
- **Trademarks:** Protect symbols, names, and logos.
- **Trade Secrets:** Protect confidential information, such as proprietary algorithms. These do not expire as long as they remain secret.

Copyright

- Copyright protects original work of authorship, including code and documentation.
- It grants the creator exclusive rights to reproduce, distribute, and create derivative works.
- In software, copyright is automatically granted when you write code. It does not protect ideas, only the expression of those ideas (e.g., the specific code you write, not the underlying algorithm).

Software Licensing Models

License compliance is critical to ensure software is used legally.

1. Proprietary Licensing: Software owned by a specific company with strict restrictions on use and modification (e.g., Microsoft 365, Adobe).

2. Open Source Licensing: Distributed under a license that allows users to access, modify, and distribute the source code.

- **Permissive (e.g., MIT, Apache):** Allows users to modify and distribute software under highly flexible terms without requiring derivatives to share the same license.
- **Copyleft (e.g., GNU GPL):** Requires any modifications or derivatives of the software to be distributed under the *exact same license*.

Help for choosing the right license: <https://choosealicense.com/>

Knowledge Check:

Scenario: You build a proprietary, closed-source web dashboard for a commercial client.

To save time, you import a highly advanced data-visualisation library from GitHub. You check the license, and it is licensed under **GPLv3** (Copyleft).

Question: What is the legal implication of this engineering decision?

Knowledge Check:

Scenario: You build a proprietary, closed-source web dashboard for a commercial client.

To save time, you import a highly advanced data-visualisation library from GitHub. You check the license, and it is licensed under **GPLv3** (Copyleft).

Question: What is the legal implication of this engineering decision?

Answer

Copyleft licenses ensure software remains open source. By linking a GPL library into your proprietary app and distributing it, your entire dashboard legally becomes subject to the GPL. You must publish all your proprietary source code for free!

Patents

Patents protect inventions, such as new software algorithms or processes. They grant the patent holder exclusive rights to use and commercialize the invention for a limited time (usually 20 years).

Examples of Software Patents:

- Google's PageRank algorithm (used in search engines).
- Amazon's 1-Click ordering system (used in e-commerce).
- Apple's slide-to-unlock feature (used in mobile devices).

Trademarks & Trade Secrets

Trademarks protect symbols, names, and logos that distinguish products or services (e.g., the Windows logo, the Apple logo).

Trade Secrets protect confidential information that provides a competitive edge. They do not expire as long as they remain secret (e.g., Google's exact search algorithm). These are protected through non-disclosure agreements (NDAs) and internal security measures.

Both are crucial for brand identity and competitive advantage in the software industry.

IP Infringement & Consequences

IP infringement occurs when someone uses, reproduces, or distributes protected intellectual property without permission. This can include copying code, using patented algorithms, or misusing trademarks.

Infringing on intellectual property can lead to severe legal consequences, including:

- **Lawsuits:** IP holders can sue for damages, which can result in costly settlements or judgments.
- **Injunctions:** Courts can order the infringing party to stop using the IP, which can disrupt business operations.

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking**
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections
- 6 Summary & Conclusion

Cybersecurity & Ethical Hacking

Cybersecurity ensures that software is secure and protects against cyber threats through secure coding, testing, and encryption.

Ethical Hacking is testing systems to identify vulnerabilities before malicious actors can exploit them. It must be conducted legally and respectfully.

- **Process:** Planning/scoping → Reconnaissance → Vulnerability scanning → Exploitation → Reporting.
- **Types:** Network penetration testing, web application testing, and Social Engineering (e.g., phishing emails to test employee awareness).
- **Hacktivism:** Using hacking as civil disobedience to promote human rights or freedom of information (e.g., WikiLeaks).

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics**
- 5 Writing Critical Reflections
- 6 Summary & Conclusion

What is (not) Decolonisation?

Decolonisation is...

- A **critical process** of questioning who defines "good" technology and whose knowledge is treated as valid.
- Redistributing power: involving local communities, languages, and lived realities in decisions.
- Changing systems (data pipelines, standards, governance), not just changing wording.

Decolonisation is *not*...

- A metaphor for generic improvement, innovation, or adding "diversity" as a checkbox.
- A one-off workshop, statement, or rebrand with no change in ownership or accountability.
- Rejecting all Western knowledge; it is about plurality, context, and equitable participation.

What is Decolonisation in Tech?

Historically, colonisation was about physical borders and resource extraction. Modern decolonisation in academia and tech addresses **Epistemic Borders** (whose knowledge, data, and culture are prioritized).

Data Colonialism (Couldry & Mejias, 2018)

The appropriation of human life through data extraction. Western tech giants extract data globally, process it centrally, and sell algorithms back to the world, often enforcing singular cultural norms on diverse populations.

<https://doi.org/10.1177/1527476418796632>

Case Study: Surveillance Tech in Palestine

This case is often discussed in decolonisation debates as an example of how digital systems can be developed and refined under conditions of occupation.

- Human rights reports document extensive biometric and predictive surveillance infrastructures used on Palestinians in the occupied Palestinian territory.
- Reported systems include facial recognition at checkpoints and large-scale data integration for movement control and risk scoring.
- **Decolonial lens:** populations with limited rights become sites where high-risk socio-technical control systems are normalized before wider diffusion.

Engineering Reflection

Ask not only *"Does it work?"* but also *"Who bears the risk, who consents, and who is accountable?"*

Beyond Data: Labour & Knowledge Colonialism

Decolonisation is not only about *data extraction*. It also includes who does the hidden work, and who gets recognition/value.

- **Labour extraction:** AI safety and moderation pipelines often rely on low-paid, high-trauma work in the Global South while value is captured by firms in the Global North.
- **Epistemic extraction:** Local language/cultural expertise is treated as raw input for models, but communities are rarely credited, paid, or given governance power.
- **Regulatory colonialism:** Firms can use weaker or under-resourced regulatory environments to scale high-risk identity/fintech systems before robust democratic oversight is in place.
- **Decolonial question:** Who creates the value, who absorbs the harm, and who owns the system?

Example (Kenya, content moderation/data labeling)

Investigations on outsourced moderation/labeling for Meta and OpenAI describe low wages, psychological harm, and asymmetric bargaining power in global AI supply chains.

Algorithmic Bias & Representation

Algorithms are not objective; they encode the biases of their creators and their datasets.

Case Study: Gender Shades (Buolamwini & Gebru, 2018)

- Commercial facial recognition systems from major tech firms had an error rate of 0.8% for light-skinned men.
- The error rate for dark-skinned women was up to 34.7%.
- **The SE Failure:** The engineering teams and their training datasets were not representative of the global population. They optimized the software for people who looked like them.

Explainable AI (XAI)

As AI is used in healthcare and law enforcement, transparency is an ethical necessity.

Explainable AI (XAI) refers to the ability of AI systems to explain their decision-making processes in a way humans can easily understand.

- **Rule-based systems:** Decision-making is based on predefined logic, making it easily explained.
- **Model transparency:** Making the inner workings more visible, explaining how specific features are used to make predictions.
- **Interactive systems:** Allowing users to ask questions and receive real-time explanations to identify potential biases.

Interactive Task: Spot the Bias

Scenario: You are designing a smart-city autonomous delivery robot. The robot uses pedestrian-detection AI trained entirely on dashcam footage from California. The robot is being deployed in rural India.

Discuss (2 mins): What are the potential catastrophic failures of this deployment from a Decolonisation/Bias perspective?

Interactive Task: Spot the Bias

Scenario: You are designing a smart-city autonomous delivery robot. The robot uses pedestrian-detection AI trained entirely on dashcam footage from California. The robot is being deployed in rural India.

Discuss (2 mins): What are the potential catastrophic failures of this deployment from a Decolonisation/Bias perspective?

Debrief

- **Visual Bias:** The AI might not recognize traditional Indian clothing (e.g., saris, dhotis) as "pedestrians."
- **Environmental Bias:** Unfamiliarity with livestock on roads, different types of vehicles (tuk-tuks), or different traffic norm behaviors.

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections**
- 6 Summary & Conclusion

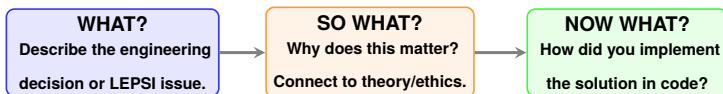
The Final Hurdle: Critical Reflection

As professional engineers, you must be able to articulate *why* you made specific technical choices.

Many junior engineers write reflections like a *diary* ("First I used React, then I built the API, it was hard."). A professional critical reflection analyzes technical choices through the lens of LEPSI and Software Engineering theory.

Borton's Model of Reflection (1970)

Use this academic framework to structure your reflective writing.



Example: Borton's Model in Action

- **[WHAT]:** During development, I decided to implement strict Role-Based Access Control (RBAC) to restrict access to sensitive endpoints.
- **[SO WHAT]:** This addresses a critical LEPSI security concern. An autonomous system is a "dual-use" technology. If the system lacks authentication, it becomes vulnerable to hijacking, violating the ACM Code of Conduct regarding public safety.
- **[NOW WHAT]:** To solve this, I designed a middleware that decodes JWTs. If the payload lacks proper authorization, the system returns a 403 Forbidden status, ensuring hardware cannot be manipulated maliciously.

Possible Reflection Themes

If you are structuring a reflection on a recent project, consider these themes:

- 1 **Security vs. Usability:** How did you balance making the system secure (RBAC, tokens) while adhering to WCAG accessibility guidelines?
- 2 **Technical Debt:** What compromises did you make to hit a deadline, and how did you plan to refactor them?
- 3 **Data Privacy:** How did you apply data minimisation principles to comply with GDPR?
- 4 **Open Source Licensing:** What libraries did you use, and what are the legal implications if you commercialised the software?

Agenda

- 1 The Professional Engineer
- 2 Intellectual Property & Licensing
- 3 Security & Ethical Hacking
- 4 Decolonisation & AI Ethics
- 5 Writing Critical Reflections
- 6 Summary & Conclusion**

Module Wrap-Up

Over this module, you have transitioned from writing scripts to engineering software.

- You gathered requirements and designed architectures.
- You built full-stack applications.
- You isolated environments using Docker containerisation.
- You automated testing and delivery using CI/CD.
- You learned to clean your code and consider the ethical weight of what you build.

You are now equipped to operate as Professional Software Engineers.

Academic References & Further Reading

- **Decolonisation & Data:** Couldry, N., & Meijas, U. A. (2019). *The Costs of Connection: How Data is Colonizing Human Life and Appropriating It for Capitalism*. Stanford University Press.
- **Palestine Surveillance Case:** Amnesty International (2023). *Automated Apartheid: How Facial Recognition Fragments, Segregates and Controls Palestinians in the OPT*.
<https://www.amnesty.org/en/latest/campaigns/2023/05/automated-apartheid/>
- **Occupation & Control Framework:** Human Rights Watch (2021). *A Threshold Crossed: Israeli Authorities and the Crimes of Apartheid and Persecution*. <https://www.hrw.org/report/2021/04/27/threshold-crossed/israeli-authorities-and-crimes-apartheid-and-persecution>
- **Digital Sovereignty & Worldcoin:** Mutung'u, G., Martin, A., & Brewczyńska, M. (2026). *Regulatory entrepreneurship's threat to digital sovereignty: the case of Worldcoin in Kenya*. *Science and Public Policy*, 53(2), 289–299. <https://doi.org/10.1093/scipol/scag010>
- **AI Labour Extraction (Meta):** Perrigo, B. (2022). *Inside Facebook's African Sweatshop*. TIME.
<https://time.com/6147458/facebook-africa-content-moderation-employee-treatment/>
- **AI Labour Extraction (OpenAI):** Perrigo, B. (2023). *OpenAI Used Kenyan Workers on Less Than \$2 Per Hour to Make ChatGPT Less Toxic*. TIME.
<https://time.com/6247678/openai-chatgpt-kenya-workers/>
- **Algorithmic Bias:** Buolamwini, J., & Gebru, T. (2018). *Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification*. *Proceedings of Machine Learning Research*.
- **Reflective Practice:** Borton, T. (1970). *Reach, Touch, and Teach*. McGraw-Hill.
- **Professional Standards:** BCS / ACM Code of Ethics.
- **Information Security:** Merkow, M. S., & Breithaupt, J. *Information Security: Principles and Practices*.

Any Questions?

`fdelduchetto@lincoln.ac.uk`